

Diagrammes UML -- Plateforme Sécurité Routière

Coller le code Mermaid dans mermaid.live pour exporter en PNG/SVG.

1. Diagramme de Cas d'Utilisation (Use Case)

```
graph TB
    subgraph Visiteur["? Visiteur (non authentifié)"]
        V_REG["S'inscrire (Local)"]
        V_LOGIN["Se connecter (JWT)"]
        V_GOOGLE["Se connecter via Google OAuth2"]
        V_FORGOT["Mot de passe oublié"]
        V_RESET["Réinitialiser mot de passe"]
        V_COURS_PUB["Consulter cours publiés"]
    end

    subgraph Apprenant["? Apprenant"]
        A_DASH["Consulter dashboard"]
        A_COURS["Consulter les cours"]
        A_QCM["Passer un QCM"]
        A_RESULT["Voir résultats QCM"]
        A_CHAT["Poser une question au Chatbot (RAG)"]
        A_IMG["Détecter un panneau par image (Vision IA)"]
        A_STT["Utiliser la reconnaissance vocale (STT)"]
        A_TTS["Écouter les réponses du bot (TTS)"]
        A_HIST["Gérer historique conversations"]
        A_FEED["Donner un feedback ??"]
        A_PROFIL["Modifier profil / mot de passe"]
        A_STATS["Voir statistiques personnelles"]
    end

    subgraph Formateur["??? Formateur"]
        F_DASH["Consulter dashboard"]
        F_COURS_C["Créer un cours"]
        F_COURS_M["Modifier / Supprimer un cours"]
        F_COURS_P["Publier / Dépublier un cours"]
        F_STATS["Voir statistiques (cours)"]
        F_PROFIL["Modifier profil"]
    end

    subgraph Admin["?? Administrateur"]
        AD_DASH["Consulter dashboard"]
        AD_USERS["Gérer les utilisateurs (CRUD)"]
        AD_ROLE["Modifier rôle utilisateur"]
        AD_STATUS["Activer / Désactiver un compte"]
        AD_QCM_C["Créer un QCM"]
        AD_QCM_M["Modifier / Supprimer un QCM"]
        AD_QCM_P["Publier un QCM"]
        AD_COURS["Gérer tous les cours"]
        AD_STATS["Voir statistiques globales"]
        AD_FEED["Consulter feedback chatbot"]
    end
```

```

    AD_INGEST["Ingérer des documents (RAG)"]
end

subgraph Systeme["?? Système"]
    S_RAG["Pipeline RAG (Pinecone + Groq LLM)"]
    S_VISION["Détection panneau (Groq Vision)"]
    S_JWT["Authentification JWT"]
    S_LANG["Détection langue (FR/AR)"]
    S_EMBED["Embeddings HuggingFace"]
end

A_CHAT --> S_RAG
A_CHAT --> S_LANG
A_IMG --> S_VISION
S_RAG --> S_EMBED
V_LOGIN --> S_JWT
V_GOOGLE --> S_JWT
AD_INGEST --> S_EMBED

```

2. Diagramme de Classes

```

classDiagram
    direction TB

    class User {
        +int id
        +String username
        +String email
        +String hashed_password
        +String full_name
        +String avatar_url
        +String auth_provider
        +String google_id
        +UserRole role
        +Boolean is_active
        +DateTime created_at
        +DateTime updated_at
    }

    class UserRole {
        <<enumeration>>
        admin
        formateur
        apprenant
    }

    class Conversation {
        +int id
        +int user_id
        +String title
        +String language
        +DateTime created_at
        +DateTime updated_at
    }

```

```

}

class Message {
    +int id
    +int conversation_id
    +String role
    +String content
    +String language
    +DateTime created_at
}

class Course {
    +int id
    +String title
    +String description
    +String category
    +CourseLevel level
    +String duration
    +String image_url
    +Text content
    +String video_url
    +Boolean is_published
    +int order
    +int created_by
    +DateTime created_at
    +DateTime updated_at
}

class CourseLevel {
    <<enumeration>>
    debutant
    intermediaire
    avance
}

class QCM {
    +int id
    +String title
    +String description
    +String category
    +String difficulty
    +int duration_minutes
    +int pass_score
    +Boolean is_published
    +int created_by
    +DateTime created_at
    +DateTime updated_at
}

class Question {
    +int id
    +int qcm_id
    +String text
    +String image_url
    +String explanation
    +int order
}

```

```

class Answer {
    +int id
    +int question_id
    +String text
    +Boolean is_correct
}

class UserQCMResult {
    +int id
    +int user_id
    +int qcm_id
    +Float score
    +int total_questions
    +int correct_answers
    +Boolean passed
    +int duration_seconds
    +String answers_detail
    +DateTime completed_at
}

class Feedback {
    +int id
    +String session_id
    +String question
    +String answer
    +String language
    +Boolean is_positive
    +String comment
    +DateTime created_at
}

User "1" --> "0..*" Conversation : possède
User "1" --> "0..*" UserQCMResult : passe
User "1" --> "0..*" Course : crée
User ..> UserRole : a un rôle

Conversation "1" --> "0..*" Message : contient

Course ..> CourseLevel : a un niveau

QCM "1" --> "1..*" Question : contient
QCM "1" --> "0..*" UserQCMResult : évalué par
User "1" --> "0..*" QCM : crée

Question "1" --> "2..*" Answer : propose

UserQCMResult --> User : apprenant
UserQCMResult --> QCM : résultat de

```

3. Diagramme de Séquence -- Chatbot RAG (Question/Réponse)

sequenceDiagram

```

autonumber
actor U as Apprenant
participant FE as Frontend Angular
participant API as FastAPI Backend
participant LD as LanguageDetector
participant PC as Pinecone (Vector Store)
participant HF as HuggingFace (Embeddings)
participant LLM as Groq LLM
participant DB as MySQL

U->>FE: Saisir question (texte ou micro STT)
FE->>API: POST /api/chat/ask {question, conversation_id}
API->>DB: Vérifier token JWT (authentification)
DB-->>API: Utilisateur valide

API->>LD: detect_language(question)
LD-->>API: langue = "fr" ou "ar"

API->>HF: encode(question) ? embedding
HF-->>API: vecteur [384 dims]

API->>PC: query(embedding, top_k=4, filter=langue)
PC-->>API: documents pertinents (contexte)

API->>LLM: prompt(contexte + question)
LLM-->>API: réponse générée

API->>DB: Sauvegarder Message(user) + Message(assistant)
DB-->>API: OK

API-->>FE: {answer, language, sources_count, signs[]}]
FE-->>U: Afficher réponse + bouton TTS ?

```

4. Diagramme de Séquence -- Authentification (Local + Google OAuth2)

```

sequenceDiagram
    autonumber
    actor U as Utilisateur
    participant FE as Frontend Angular
    participant API as FastAPI Backend
    participant DB as MySQL
    participant G as Google OAuth2

    alt Inscription locale
        U->>FE: Remplir formulaire (username, email, password)
        FE->>API: POST /api/auth/register
        API->>API: Valider données + hash bcrypt
        API->>DB: INSERT User(role=apprenant)
        DB-->>API: User créé
        API->>API: Générer access_token + refresh_token (JWT HS256)
        API-->>FE: {access_token, refresh_token, user}
    end

```

```

alt Connexion locale
  U->>FE: Saisir email + password
  FE->>API: POST /api/auth/login
  API->>API: Vérifier rate limit (5 tentatives / 5 min)
  API->>DB: SELECT User WHERE email
  DB-->>API: User trouvé
  API->>API: Vérifier bcrypt(password, hashed_password)
  API->>API: Générer JWT (access 15min + refresh 7j)
  API-->>FE: {access_token, refresh_token, user}
end

```

```

alt Connexion Google OAuth2
  U->>FE: Cliquer "Se connecter avec Google"
  FE->>G: Authentification Google
  G-->>FE: id_token Google
  FE->>API: POST /api/auth/google {token}
  API->>G: Vérifier id_token (tokeninfo)
  G-->>API: {email, name, google_id, picture}
  API->>DB: UPSERT User(auth_provider=google)
  API->>API: Générer JWT
  API-->>FE: {access_token, refresh_token, user}
end

```

```

FE->>FE: Stocker tokens localStorage
FE->>FE: Rediriger vers /{role}/dashboard

```

5. Diagramme de Séquence -- Détection de Panneau par Image (Vision IA)

```

sequenceDiagram
    autonumber
    actor U as Apprenant
    participant FE as Frontend Angular
    participant API as FastAPI Backend
    participant VISION as Groq Vision API
    participant DB as MySQL

    U->>FE: Cliquer bouton ? (upload image)
    FE->>FE: Afficher preview image dans le chat
    FE->>API: POST /api/chat/detect-sign (FormData: image, language)
    API->>DB: Vérifier token JWT
    DB-->>API: OK

    API->>API: Encoder image en base64
    API->>VISION: Envoyer image + prompt d'identification
    Note over VISION: Modèle: llama-4-scout-17b-16e-instruct
    VISION-->>API: Analyse JSON {sign_name, description, category, confidence, rules[]}

    API->>DB: Sauvegarder dans conversation (Message user + assistant)
    DB-->>API: OK

    API-->>FE: {sign_name, description, category, confidence, rules[]}
    FE-->>U: Afficher nom du panneau, catégorie, règles associées

```

6. Diagramme de Séquence -- Passer un QCM

```
sequenceDiagram
    autonumber
    actor U as Apprenant
    participant FE as Frontend Angular
    participant API as FastAPI Backend
    participant DB as MySQL

    U->>FE: Accéder à la liste des QCM
    FE->>API: GET /api/qcm/list
    API->>DB: SELECT QCM WHERE is_published=true
    DB-->>API: Liste des QCM publiés
    API-->>FE: [{id, title, category, difficulty, duration}]

    U->>FE: Choisir un QCM
    FE->>API: GET /api/qcm/{id}
    API->>DB: SELECT QCM + Questions + Answers
    DB-->>API: QCM complet
    API-->>FE: {title, questions[{text, answers[]}]}

    U->>FE: Répondre aux questions + Soumettre
    FE->>API: POST /api/qcm/{id}/submit {answers[]}
    API->>API: Calculer score + vérifier pass_score
    API->>DB: INSERT UserQCMResult
    DB-->>API: Résultat sauvé
    API-->>FE: {score, total_questions, correct_answers, passed}
    FE-->>U: Afficher résultat (réussi/échoué + score)
```